

Blackpebble — Replit Prompt: PumpFun Paper Trading Simulator + Remove PFP Generator

FIRST: Remove PFP Generator

Remove the "Institutional Profile Generator" / "Coming Soon" PFP section from the Community page entirely. Delete all related code and components. There is no mascot so this feature doesn't apply.

MAIN FEATURE: Paper Trading Simulator

Add a full paper trading simulator to the Blackpebble website. This is a new page called "Trading Desk" accessible from the main navigation. It allows users to practice trading real PumpFun/Solana tokens with fake SOL — real prices, fake money. The data must be 100% accurate using live market prices.

Brand this as: **"Blackpebble Trading Desk — Shareholder Training Program"**

The corporate framing: "Before deploying real capital, all prospective shareholders are encouraged to demonstrate conviction through our risk-free simulation environment."

NAVIGATION UPDATE

Add "Trading Desk" to the main navigation bar, between "Insights" and "Community":

About Us | Investment Strategy | The Vault | Insights | **Trading Desk** | Community

PAGE: TRADING DESK

Hero Section

- Headline: "Trading Desk"
- Subtitle: "Shareholder Training Program — Demonstrate conviction in a risk-free environment."
- Below subtitle: "Trade real tokens at real prices with simulated capital. Top performers earn enhanced distribution weight."

Connect Wallet Prompt

- If wallet not connected: Show "Connect Wallet to Begin" button (same Phantom wallet adapter already on the site)
- Once connected, the user's paper trading account is tied to their wallet address
- New accounts start with **100 SOL** paper balance

TRADING INTERFACE

Layout (Desktop):

Plain Text

[Search Token]

Paper Balance: 100 SOL

TOKEN INFO PANEL

- Name, ticker, logo

- Current price

- Market cap

- 24h volume

- 24h change %

- Holder count

- Price chart (mini)

TRADE PANEL

- Buy / Sell tabs

- Amount input (SOL)

- Estimated tokens received

- Execute Trade button

OPEN POSITIONS

Token | Entry Price | Current Price | P&L % | Value | [Sell]

TRADE HISTORY

Token | Side | Price | Amount | P&L | Date

LEADERBOARD (Top 50)

Rank | Wallet | Total P&L | Win Rate | Trades | ROI %

Layout (Mobile):

- Stack everything vertically
 - Search bar at top
 - Token info condensed to one row (name, price, 24h%)
 - Trade panel below
 - Tabs for: Positions | History | Leaderboard (swipeable or tab buttons)
-

TOKEN SEARCH & PRICE DATA

Search Functionality:

- Search bar at top of page
- User can search by token name, ticker, or contract address
- Search queries DexScreener API for matching tokens on Solana
- Show top 5-10 results in a dropdown with: name, ticker, price, 24h change, market cap

Price Data Sources (ALL FREE, NO API KEY NEEDED):

DexScreener API — Token Search & Metadata:

Plain Text

```
GET https://api.dexscreener.com/latest/dex/search?q={query}
```

Returns: token name, symbol, price, market cap, volume, price change, pair address, logo URL

DexScreener API — Token by Address:

Plain Text

```
GET https://api.dexscreener.com/latest/dex/tokens/{tokenAddress}
```

Returns: all pairs for that token with full price data

Jupiter Price API — Real-time Price:

Plain Text

```
GET https://price.jup.ag/v6/price?ids={mintAddress}
```

Returns: current price in USD, very fast, very accurate

Use DexScreener for: search, metadata, charts, market cap, volume, holder info

Use Jupiter for: execution price when calculating trades (most accurate spot price)

Price Update Frequency:

- When viewing a token: poll Jupiter price every 10 seconds
- Open positions: update P&L every 30 seconds
- Leaderboard: recalculate every 5 minutes

TRADE EXECUTION LOGIC

Buying:

Plain Text

User inputs: X SOL to spend on Token Y
Current price: fetched from Jupiter API at moment of execution
Tokens received: $X \text{ SOL} / \text{token_price_in_SOL}$
Fee: simulate 1% slippage (deduct from tokens received)

Store in database:

JSON

```
{
  "tradeId": "uuid",
  "wallet": "user_wallet_address",
  "tokenMint": "token_contract_address",
  "tokenName": "TOKENNAME",
  "side": "buy",
  "solAmount": 5.0,
  "tokenAmount": 250000,
  "pricePerToken": 0.00002,
  "timestamp": "2025-01-15T10:30:00Z"
}
```

Selling:

Plain Text

User selects position to sell (full or partial)
Current price: fetched from Jupiter API
SOL received: $\text{tokens_sold} * \text{current_price_in_SOL}$
Fee: simulate 1% slippage (deduct from SOL received)
P&L calculated: SOL received - SOL originally spent (proportional if partial se

Rules:

- Cannot spend more than paper balance
- Cannot sell more tokens than held
- Minimum trade: 0.1 SOL
- Maximum open positions: 20 (prevents spam)
- Trades are instant (no waiting for "confirmation" — it's paper trading)
- After trade executes: show brief confirmation toast ("Bought 250,000 TOKEN for 5 SOL")

PORTFOLIO & P&L TRACKING

Open Positions Display:

For each position show:

Token	Avg Entry	Current Price	Quantity	Value (SOL)	P&L (SOL)	P&L %	Action
TOKEN	0.00002	0.00003	250,000	7.5 SOL	+2.5 SOL	+50%	[Sell]

- P&L color: green if positive, red if negative
- Update current price every 30 seconds
- "Value" = current token amount $\times$ current price

Portfolio Summary (top of positions section):

Plain Text

Paper Balance: 45.2 SOL (available to trade)
Portfolio Value: 87.3 SOL (open positions value)

Total Value: 132.5 SOL (balance + positions)
Total P&L: +32.5 SOL (+32.5%)

Trade History:

Date	Token	Side	Amount	Price	P&L
Jan 15	TOKEN	Buy	5 SOL → 250K tokens	0.00002	—
Jan 16	TOKEN	Sell	250K tokens → 7.5 SOL	0.00003	+2.5 SOL

Stats Card:

- Total Trades: X
 - Win Rate: X% (trades closed in profit / total closed trades)
 - Best Trade: +X SOL (token name)
 - Worst Trade: -X SOL (token name)
 - Average Hold Time: X hours
 - Current Streak: X wins in a row
-

LEADERBOARD

Global Leaderboard (visible to everyone):

Rank	Wallet	Total P&L (SOL)	ROI %	Win Rate	Trades
1	7xKX...gAsU	+142.5 SOL	+142.5%	78%	34
2	9aE4...kWcX	+98.2 SOL	+98.2%	65%	52
...					

Leaderboard Rules:

- Ranked by Total P&L (realized + unrealized)

- Minimum 5 closed trades to appear on leaderboard
- Show top 50
- Wallet addresses truncated (first 4...last 4)
- If user is connected, highlight their row
- "Your Rank: #27" shown at top if they're on the board

Time Periods:

- Tabs: All Time | This Month | This Week
 - Weekly resets for the weekly competition (see Airdrop Integration below)
-

AIRDROP INTEGRATION

This is the key differentiator — paper trading performance affects real \$BLK airdrop allocations.

Participation Score Boost:

- Simply using the simulator adds to the user's "Participation Score" in the airdrop qualification framework
- 1 point per trade executed (capped at 5 points per day to prevent spam)
- Points stored in the same database as the airdrop eligibility system

Weekly Competition:

- Every Monday at 00:00 UTC, weekly P&L resets for competition purposes (all-time stats preserved)
- Top 10 weekly performers earn a **1.25x multiplier** on their next airdrop allocation
- Top 3 earn **1.5x multiplier**
- #1 earns **2.0x multiplier**
- Multiplier applies to the NEXT distribution only, then resets
- Display on leaderboard: "Weekly Competition — Ends in: 3d 14h 22m"

Graduation Tiers:

Track cumulative all-time paper P&L. When users hit milestones, they earn permanent perks:

Milestone	Reward
+50 SOL all-time P&L	"Analyst" badge — permanent +10% airdrop weight
+200 SOL all-time P&L	"Senior Analyst" badge — permanent +20% airdrop weight
+500 SOL all-time P&L	"Fund Manager" badge — permanent +30% airdrop weight

Display on Wallet Connect (Vault page):

When a user connects their wallet on the Vault page "Check Your Allocation" section, also show:

- Paper Trading Rank: #X
- Weekly Competition Status: "Top 10 — 1.25x bonus active" or "Not qualified"
- Graduation Tier: "Analyst (+10%)" or "No tier yet — need +50 SOL P&L"

Anti-Gaming Rules:

- Maximum 20 open positions (prevents opening 100 positions hoping one moons)
- Minimum 0.1 SOL per trade (prevents dust trades for point farming)
- 5 trades per day count toward participation score (more trades are allowed but don't give extra points)
- Accounts that only buy and never sell don't count toward leaderboard (must have 5+ closed trades)

BACKEND REQUIREMENTS

Database Schema (add to existing SQLite):

SQL

```
CREATE TABLE IF NOT EXISTS paper_accounts (
  wallet TEXT PRIMARY KEY,
  paper_balance REAL DEFAULT 100.0,
  total_pnl REAL DEFAULT 0.0,
  realized_pnl REAL DEFAULT 0.0,
```

```

total_trades INTEGER DEFAULT 0,
winning_trades INTEGER DEFAULT 0,
participation_points INTEGER DEFAULT 0,
graduation_tier TEXT DEFAULT 'none',
created_at TEXT,
last_trade_at TEXT
);

CREATE TABLE IF NOT EXISTS paper_positions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  wallet TEXT NOT NULL,
  token_mint TEXT NOT NULL,
  token_name TEXT,
  token_symbol TEXT,
  token_logo TEXT,
  total_tokens REAL NOT NULL,
  total_sol_spent REAL NOT NULL,
  avg_entry_price REAL NOT NULL,
  opened_at TEXT NOT NULL,
  status TEXT DEFAULT 'open',
  FOREIGN KEY (wallet) REFERENCES paper_accounts(wallet)
);

CREATE TABLE IF NOT EXISTS paper_trades (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  wallet TEXT NOT NULL,
  token_mint TEXT NOT NULL,
  token_name TEXT,
  token_symbol TEXT,
  side TEXT NOT NULL,
  sol_amount REAL NOT NULL,
  token_amount REAL NOT NULL,
  price_per_token REAL NOT NULL,
  pnl_sol REAL DEFAULT 0,
  timestamp TEXT NOT NULL,
  FOREIGN KEY (wallet) REFERENCES paper_accounts(wallet)
);

CREATE TABLE IF NOT EXISTS weekly_competitions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  wallet TEXT NOT NULL,
  week_start TEXT NOT NULL,
  week_pnl REAL DEFAULT 0.0,
  week_trades INTEGER DEFAULT 0,
  rank INTEGER,
  multiplier_earned REAL DEFAULT 1.0
);

```

```
CREATE INDEX IF NOT EXISTS idx_positions_wallet ON paper_positions(wallet);
CREATE INDEX IF NOT EXISTS idx_trades_wallet ON paper_trades(wallet);
CREATE INDEX IF NOT EXISTS idx_weekly_wallet ON weekly_competitions(wallet);
```

API Endpoints:

GET /api/paper/account/:wallet

Returns user's paper trading account info (balance, stats, tier)

POST /api/paper/search

Body: { "query": "token name or address" }

Proxies to DexScreener search API, returns formatted results

GET /api/paper/token/:mintAddress

Returns full token info + current price (proxies DexScreener + Jupiter)

POST /api/paper/trade

Body: { "wallet": "...", "tokenMint": "...", "side": "buy|sell", "amount": 5.0, "positionId": "..." }

Executes paper trade, updates balances and positions

GET /api/paper/positions/:wallet

Returns all open positions with current prices and P&L

GET /api/paper/history/:wallet

Returns trade history for the wallet

GET /api/paper/leaderboard?period=all|month|week

Returns top 50 ranked by P&L for the given period

GET /api/paper/competition

Returns current weekly competition status (time remaining, top performers)

Cron Jobs:

JavaScript

```
// Update all open position prices every 2 minutes (for leaderboard accuracy)
cron.schedule('* /2 * * * *', updateOpenPositionPrices);

// Reset weekly competition every Monday 00:00 UTC
cron.schedule('0 0 * * 1', resetWeeklyCompetition);

// Check graduation tier milestones every hour
cron.schedule('0 * * * *', checkGraduationMilestones);
```

Price Fetching (server-side to avoid CORS):

JavaScript

```
async function getTokenPrice(mintAddress) {
  try {
    // Primary: Jupiter
    const jupResponse = await axios.get(
      `https://price.jup.ag/v6/price?ids=${mintAddress}`
    );
    if (jupResponse.data.data[mintAddress]) {
      return jupResponse.data.data[mintAddress].price;
    }

    // Fallback: DexScreener
    const dexResponse = await axios.get(
      `https://api.dexscreener.com/latest/dex/tokens/${mintAddress}`
    );
    if (dexResponse.data.pairs && dexResponse.data.pairs.length > 0) {
      return parseFloat(dexResponse.data.pairs[0].priceUsd);
    }

    return null;
  } catch (error) {
    console.error(`Price fetch failed for ${mintAddress}:`, error.message);
    return null;
  }
}

async function searchTokens(query) {
  try {
    const response = await axios.get(
      `https://api.dexscreener.com/latest/dex/search?q=${encodeURIComponent(query)}`
    );
    // Filter to Solana only
    const solanaPairs = response.data.pairs.filter(p => p.chainId === 'solana')
    // Deduplicate by token address, take highest liquidity pair
    const seen = new Map();
    for (const pair of solanaPairs) {
      const mint = pair.baseToken.address;
      if (!seen.has(mint) || pair.liquidity.usd > seen.get(mint).liquidity.usd) {
        seen.set(mint, pair);
      }
    }
    return Array.from(seen.values()).slice(0, 10).map(pair => ({
      mint: pair.baseToken.address,
      name: pair.baseToken.name,
    }));
  }
}
```

```

        symbol: pair.baseToken.symbol,
        price: parseFloat(pair.priceUsd),
        priceChange24h: pair.priceChange?.h24 || 0,
        marketCap: pair.marketCap || 0,
        volume24h: pair.volume?.h24 || 0,
        logo: pair.info?.imageUrl || null
      }));
    } catch (error) {
      console.error('Token search failed:', error.message);
      return [];
    }
  }
}

```

UI STYLING

Match the existing Blackpebble dark theme exactly:

- Background: #0a0a0a
- Cards/panels: #141414 with border: 1px solid #1f1f1f
- Text: white (#ffffff) for primary, #a0a0a0 for secondary
- Accent: #c9a96e (gold) for highlights, active states, positive P&L headers
- Positive P&L: #22c55e (green)
- Negative P&L: #ef4444 (red)
- Headings: Playfair Display
- Body/data: Inter
- Monospace for numbers/prices: JetBrains Mono or Courier New

Trade Button Styling:

- Buy button: outlined gold border, gold text. On hover: filled gold, black text
- Sell button: outlined red border, red text. On hover: filled red, white text

Leaderboard Styling:

- Top 3 rows have subtle gold/silver/bronze left border accent
- Connected user's row highlighted with subtle gold background tint
- Rank numbers in gold for top 10

Loading States:

- Skeleton pulse animations while fetching prices
- "Searching..." indicator in search dropdown
- Brief loading spinner on trade execution (500ms minimum to feel intentional)

Empty States:

- No positions: "No open positions. Search for a token above to make your first trade."
 - No history: "No trades yet. Your trading history will appear here."
 - Not connected: Full page prompt to connect wallet
-

MOBILE RESPONSIVENESS

Critical — must work perfectly on iPhone (375px - 430px width):

- Search bar full width
 - Token info stacks vertically (name/price on one line, stats below)
 - Trade panel full width below token info
 - Amount input large enough for thumb tapping
 - Positions/History/Leaderboard as horizontal tab buttons (not side by side)
 - Table rows become cards on mobile (stacked info, not columns)
 - P&L numbers large and readable
 - Buy/Sell buttons full width, 48px height minimum (touch target)
-

ERROR HANDLING

- If Jupiter/DexScreener API is down: show "Price unavailable" instead of the price, disable trading for that token
 - If a token is delisted or has no liquidity: show "Token no longer tradeable" and allow user to close position at last known price
 - If database write fails: show error toast "Trade failed — please try again" and don't deduct balance
 - Never show raw error messages, stack traces, or "undefined" to users
 - If wallet disconnects mid-session: preserve all data, just show "Connect Wallet" prompt again
-

IMPORTANT NOTES

- Keep ALL existing pages and features intact. This is an ADDITION.
- The Trading Desk is a new page/route: /trading
- Reuse the existing Phantom wallet connection (don't create a second wallet adapter instance)
- All price data is fetched server-side and served to frontend via API (avoids CORS issues with DexScreener/Jupiter)
- Paper balance resets are NOT automatic — users keep their 100 SOL unless they lose it all. If they hit 0, show "Account depleted. Reset available in: X days" (allow reset once per week)
- The weekly competition multiplier data should be accessible by the existing airdrop eligibility system (same database)
- Install additional packages as needed: uuid (for trade IDs), node-cron (already installed)